

LeetCode 1823: Winner of the Circular Game

Authored by Arpit Raj, LNMIIT Jaipur | July 14, 2026

1. Rules of the Game

Given n friends and a step count k , the game proceeds as follows:

1. Start at the 1st friend.
2. Count the next k friends including the friend you started at.
3. The last counted friend leaves the circle.
4. If more than 1 survivor remains, continue the game. The count starts at the next friend immediately following the one who just lost.
5. Output the last surviving member.

2. Approach 1: Queue Simulation

This approach physically simulates the elimination process using a queue data structure.

Algorithm

- **Step 1:** The first $(k - 1)$ people are safe and have skipped elimination for the current round. Move them to the back of the queue.
- **Step 2:** Eliminate the k^{th} person by popping them from the queue.
- **Step 3:** Repeat until only one person remains in the queue.

Time Complexity: $O(n \times k)$ Every round, $k - 1$ players are pushed back, and 1 player is eliminated. This happens for $n - 1$ rounds.

Space Complexity: $O(n)$ To store the n players in the queue.

C++ Implementation

```
class Solution {
public:
    int findTheWinner(int n, int k) {

        queue<int> q;
        for(int i=1;i<=n;i++){
            q.push(i);
        }

        while(q.size()>1){

            for(int i=0;i<k-1;i++){
                q.push(q.front());
                q.pop(); //moving k-1 survivors to back
            }

            q.pop(); //eliminating the loser
        }
        return q.front();
    }
};
```

3. Approach 2: Mathematical / Recursive

We can optimize the solution by recognizing the pattern in how the index shifts after every elimination.

Logic

- For a circle of size 1, the winner is obviously the person at index 0.
- When we increase the circle size, everyone's relative position shifts by k indices because every elimination rotates the circle by k .

$$\text{new winner} = (\text{old winner} + k) \% n$$

Since this math inherently uses 0-based indexing for the modulo arithmetic to work seamlessly, we must add 1 at the end to convert the final answer back to 1-based indexing.

Time Complexity: $O(n)$

We only perform n recursive calls.

Space Complexity: $O(n)$

For the recursion stack. (Note: The iterative version of this logic reduces the space complexity to $O(1)$).

C++ Implementation

```
class Solution {
public:

    int recurse(int n,int k){
        if(n==1){
            return 0;
        }

        return (recurse(n-1,k)+k)%n;
    }
    int findTheWinner(int n, int k) {
        return recurse(n,k)+1;
    }
};
```